REQ1 Design Rationale

Weight-Limited Inventory

The Inventory class is abstract with add() and remove() as extension points. We decided not to modify the engine, so we extended it with WeightLimitedInventory, overriding add() to ensure the current weight is below a threshold. This centralises the weight handling and doesn't modify the engine. All items must have an ItemStatistics.WEIGHT statistic so there is always a value stored in the inventory that can be used without knowing what the item is.

Item-Declared Actions (No instanceof in ContractedWorker)

Originally, ContractedWorker.playTurn() would have to inspect each item using a series of instanceof tests to determine allowable actions. This was deemed unacceptable because every new item type would require changes to ContractedWorker, making it hard to decouple. Instead, items override allowableActions(Actor owner, GameMap map) to return the appropriate actions. ContractedWorker just displays a menu - it doesn't know about item types. This complies with the Open/Closed principle: the actor class is not modified. The engine's World.These are already automatically collected by prepareAllowableActions(), so the original loops that were added to ContractedWorker have been removed.

Door Encapsulation

The isUnlocked field was changed to private with unlock() and isUnlocked() methods. This ensures no class can set the field to anything it wants and the locking contract is defined. UnlockDoorAction just calls door.unlock() - it doesn't need to know how the door is implemented.

AccessCard Scanning Exits

AccessCard checks the exits of the actor's location when allowableActions() is called. So the unlock option is only visible if a locked door is nearby. This means no observer is required - the check is lazy and on-demand.

Flask Gets Non-Portable when Empty

When the last charge of a Flask is used, makeNonPortable() is called on it. This prevents the game from providing a drop action, so that the empty flask remains in the inventory rather than the menu being cluttered with an indroppable husk. It continues to contribute to the weight of the actor, punishing him or her for carrying a useless item.

FirstAidKit Cooldown Only While Carried

Item.tick(Location, Actor) is called when the item is carried, while Item.tick(Location) is called when on the ground. The code for the cooldown is in the overload for carried, so the timer stops the instant a worker drops the kit. This is as per the spec and no change to the engine was needed.

UseFirstAidKitAction and increaseMaximum

modifyStatisticMaximum with INCREASE calls BaseStatistic.increaseMaximum(), which sets the current value to the maximum value as a side effect. So the call increases the maximum and heals the actor in one go; there's no need for a separate heal() call.